Bronze layer - weather ingestion

Fetch recent FMI weather observations and append them to the Bronze layer. This notebook keeps weather events close to source format and applies only minimal parsing and timestamp normalization before writing to `bronze_events`.

In [ ]:
```
from pyspark.sql import functions as F
import requests
import xml.etree.ElementTree as ET
import uuid
from datetime import datetime, timedelta, timezone

_ = spark.sql("USE azure_streaming_mvp")
```

Processing parameters

The parameters below control the FMI endpoint, request scope, and default lookback window for recent observations.

In [ ]:
```
FMI_WFS = "https://opendata.fmi.fi/wfs"

DEFAULT_PLACE = "helsinki"
DEFAULT_PARAMS = "t2m"
DEFAULT_LOOKBACK_MINUTES = 360
```

FMI request helper

Fetch recent weather observations from the FMI WFS endpoint using the timevaluepair stored query.

In [ ]:
```
def fetch_fmi_timevaluepair(
    place: str = DEFAULT_PLACE,
    params: str = DEFAULT_PARAMS,
    minutes: int = DEFAULT_LOOKBACK_MINUTES
    ) -> str:
    """
```

```
    Fetch recent FMI observations (WFS timevaluepair XML).

    Args:
        place: Location name used by FMI (default: "helsinki").
        params: Comma-separated FMI parameter names (e.g. "t2m", "ws_10min").
        minutes: Lookback window in minutes.

    Returns:
        Raw XML response text from FMI WFS.
    """
    now = datetime.now(timezone.utc)
    start = now - timedelta(minutes=minutes)

    query_params = {
        "service": "WFS",
        "version": "2.0.0",
        "request": "getFeature",
        "storedquery_id": "fmi::observations::weather::timevaluepair",
        "place": place,
        "parameters": params,
        "starttime": start.strftime("%Y-%m-%dT%H:%M:%SZ"),
        "endtime": now.strftime("%Y-%m-%dT%H:%M:%SZ")
    }

    response = requests.get(FMI_WFS, params=query_params, timeout=60)
    response.raise_for_status()
    return response.text
```

FMI XML parsing

Parse the FMI WFS XML response into Bronze-compatible weather event rows.

In [ ]:
```
def parse_fmi_events(xml_text: str, place: str = DEFAULT_PLACE) -> list[dict]:
    """
    Parse FMI WFS timevaluepair XML into event rows.

    Extracts (time, value) points and attaches minimal station metadata
    (id, name, lat/lon) when available.
```

```
    Args:
        xml_text: Raw XML text returned by FMI WFS.
        place: Place name used for the request context (fallback id).

    Returns:
        List of dict rows compatible with the Bronze schema.
    """
    ns = {
        "wfs": "http://www.opengis.net/wfs/2.0",
        "gml": "http://www.opengis.net/gml/3.2",
        "om": "http://www.opengis.net/om/2.0",
        "wml2": "http://www.opengis.net/waterml/2.0",
        "target": "http://xml.fmi.fi/namespace/om/atmosphericfeatures/1.1",
        "xlink": "http://www.w3.org/1999/xlink",
    }

    def safe_text(node):
        if node is None or node.text is None:
            return None
        return node.text.strip()

    def xlink_href(node):
        if node is None:
            return None
        return node.attrib.get("{" + ns["xlink"] + "}href")

    def infer_metric(href):
        if not href:
            return None
        if "param=" in href:
            tail = href.split("param=", 1)[1] # split once, in href &amp;lt;=&amp;gt; &
            return tail.split("&", 1)[0]
        return None

    root = ET.fromstring(xml_text)

    events = []

    for member in root.findall(".//wfs:member", ns):
        # Extract metric
        observed = member.find(".//om:observedProperty", ns)
```

```
        metric = infer_metric(xlink_href(observed)) or "t2m" # fallback

        # Station metadata
        fmisid = safe_text(member.find(".//gml:identifier", ns))
        station_name = safe_text(member.find(".//gml:Point/gml:name", ns)) or safe_text(member.find(".//gml:name", ns))
        region = safe_text(member.find(".//target:region", ns))

        pos_text = safe_text(member.find(".//gml:Point/gml:pos", ns))   # "lat lon"
        lat, lon = None, None
        if pos_text:
            parts = pos_text.split()
            if len(parts) >= 2:
                lat, lon = parts[0], parts[1]

        # Time-value pairs
        for tvp in member.findall(".//wml2:MeasurementTVP", ns):
            t = safe_text(tvp.find("./wml2:time", ns))
            v = safe_text(tvp.find("./wml2:value", ns))
            if t is None or v is None:
                continue
            try:
                v_float = float(v)
            except ValueError:
                continue

            events.append({
                "event_id": str(uuid.uuid4()),
                "event_time_raw": t,     # ISO UTC string
                "source": "fmi_weather",
                "entity_type": "weather_station",
                "entity_id": fmisid or place,
                "metric": metric,
                "value": v_float,
                "unit": "C",     # MVP: t2m is Celsius
                "attrs": {
                    "place": place,
                    "station_name": station_name,
                    "region": region,
                    "lat": lat,
                    "lon": lon,
                    "fmisid": fmisid,
```

```
                    "observed_property_href": xlink_href(observed),
                }
            })
    return events
```

Bronze ingestion

Fetch recent FMI observations, parse them into the shared Bronze schema, and append them to `bronze_events`

In [ ]:
```
def ingest_fmi(
    place: str = DEFAULT_PLACE,
    params: str = DEFAULT_PARAMS,
    minutes: int = DEFAULT_LOOKBACK_MINUTES
) -> None:
    xml_text = fetch_fmi_timevaluepair(place=place, params=params, minutes=minutes)
    rows = parse_fmi_events(xml_text, place=place)

    member_count = xml_text.count("<wfs:member")
    if not rows:
        msg = (
        f"No FMI rows parsed (place={place}, params={params}, minutes={minutes}). "
        f"WFS members found: {member_count}. "
        "Next: verify storedquery/parameters and XML namespaces; "
        "if members=0, consider widening the time window."
        )
        print(msg)
        return

    df = spark.createDataFrame(rows)
    df2 = (df
           .withColumn("event_time_ts", F.to_timestamp("event_time_raw", "yyyy-MM-dd'T'HH:mm:ssX"))
           .withColumn("ingest_time_ts", F.current_timestamp())
    )

    df2.write.mode("append").saveAsTable("bronze_events")
    print(f"Appended FMI events: {len(rows)}")
```

In [ ]:
```
# Example run
ingest_fmi(place=DEFAULT_PLACE,
```

```
            params=DEFAULT_PARAMS,
            minutes=DEFAULT_LOOKBACK_MINUTES
)
```

```
Appended FMI events: 36
```

Data validation checks

These checks confirm that FMI weather events were appended successfully to the Bronze layer and that no obvious duplicate keys were introduced.

In [ ]:
```
# Check duplicate keys
spark.sql("""
SELECT metric, entity_id, event_time_ts, COUNT(*) AS row_count
FROM bronze_events
WHERE source = 'fmi_weather'
GROUP BY metric, entity_id, event_time_ts
HAVING row_count > 1
""").show(truncate=False)
```

```
+------+---------+-------------+---------+
|metric|entity_id|event_time_ts|row_count|
+------+---------+-------------+---------+
+------+---------+-------------+---------+
```

In [ ]:
```
# Preview recent FMI rows
spark.sql("""
SELECT event_time_ts,
       value,
       attrs.station_name,
       attrs.lat,
       attrs.lon
FROM bronze_events
WHERE source='fmi_weather'
ORDER BY ingest_time_ts DESC
LIMIT 10
""").show(truncate=False)
```

```
+-------------------+-----+-------------------+--------+--------+
|event_time_ts      |value|station_name       |lat     |lon     |
+-------------------+-----+-------------------+--------+--------+
|2026-03-08 11:10:00|6.4  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 10:40:00|6.1  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 10:50:00|5.9  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 11:00:00|6.1  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 10:00:00|5.6  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 10:30:00|5.7  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 09:50:00|5.3  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 10:20:00|5.7  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 10:10:00|5.5  |Helsinki Kaisaniemi|60.17523|24.94459|
|2026-03-08 11:20:00|6.4  |Helsinki Kaisaniemi|60.17523|24.94459|
+-------------------+-----+-------------------+--------+--------+
```

In [ ]:
```
# Row count
spark.sql("""
SELECT COUNT(*) AS row_count
FROM bronze_events
WHERE source='fmi_weather'
""").show()
```

```
+---------+
|row_count|
+---------+
|       36|
+---------+
```

In [ ]:
```
# Latest event time
spark.sql("""
SELECT MAX(event_time_ts) AS latest_event_time
FROM bronze_events
WHERE source = 'fmi_weather'
""").show()
```

```
+-------------------+
|  latest_event_time|
+-------------------+
|2026-03-08 15:40:00|
+-------------------+
```

In [ ]:
```
# ---- Notebook completion signal ----
dbutils.notebook.exit("OK")
```